

Table des matières

Labo 06 OLAP — Agrégation avancée et pivots sur l’entrepôt..... 1

 Partie 1 — Du GROUP BY au ROLLUP..... 1

 Partie 2 — À la carte, étiqueté, croisé 2

 Et ensuite ?..... 5

Labo 06 OLAP — Agrégation avancée et pivots sur l’entrepôt

Laboratoire du Cours 06, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

On met en pratique les sous-totaux, totaux généraux et tableaux croisés sur l’entrepôt AdventureWorks DW, dans la ligne de commande DuckDB. Depuis `Projet_SQL/`, ouvrez l’entrepôt par `duckdb adventureworksdw.duckdb`, puis posez le schéma par `USE dbo;`. Comptez une demi-heure.

Les deux voies — Ce labo a un jumeau, `Labo_06_OLTP.md`, qui pose les mêmes questions sur le schéma normalisé d’AdventureWorks dans PostgreSQL. Faites celui-ci d’abord, puis l’autre, et gardez-les ouverts côte à côte. C’est le cours où les deux voies divergent le plus, parce que PIVOT n’existe pas sur PostgreSQL.

Tables nécessaires — Ce labo croise `FactInternetSales` avec `DimDate`, `DimSalesTerritory`, et la chaîne produit `DimProduct`, `DimProductSubcategory`, `DimProductCategory`. Si l’une manque, chargez-la comme au Cours 00.

Partie 1 — Du GROUP BY au ROLLUP

Théorie : *Theorie_06.md*, sections 1 et 3.

D’abord le GROUP BY que vous connaissez, les ventes par année et par pays, un seul niveau :

```
SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN   dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear, t.SalesTerritoryCountry
ORDER BY annee, pays;
```

Maintenant le ROLLUP, les mêmes lignes, plus un sous-total par année, plus le total général :

```

SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN   dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (d.CalendarYear, t.SalesTerritoryCountry)
ORDER BY annee, pays;

```

Observez. De nouvelles lignes apparaissent, avec pays à NULL (le sous-total de l'année) et une ligne où annee et pays sont tous les deux NULL (le total général). Le détail et ses totaux, en une seule requête.

Partie 2 — À la carte, étiqueté, croisé

2.1 — GROUPING SETS et CUBE

Théorie : Theorie_06.md, sections 2 et 4.

Objectif : voir la différence entre choisir ses niveaux et tout croiser.

GROUPING SETS donne exactement les niveaux qu'on demande, ici par année, par pays, et le total :

```

SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN   dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY GROUPING SETS ((d.CalendarYear), (t.SalesTerritoryCountry), ())
ORDER BY annee, pays;

```

CUBE donne toutes les combinaisons d'un coup :

```

SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN   dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY CUBE (d.CalendarYear, t.SalesTerritoryCountry)
ORDER BY annee, pays;

```

Observez. Le CUBE ajoute, par rapport au ROLLUP de la Partie 1, les sous-totaux par pays toutes années confondues, la combinaison que la cascade ne donnait pas.

2.2 — Étiqueter les totaux avec GROUPING()

Théorie : Theorie_06.md, section 5.

Objectif : remplacer les NULL de sous-total par des libellés lisibles.

```
SELECT
    CASE WHEN GROUPING(d.CalendarYear) = 1 THEN 'Toutes années'
    ELSE CAST(d.CalendarYear AS VARCHAR) END AS annee,
    CASE WHEN GROUPING(t.SalesTerritoryCountry) = 1 THEN 'Tous pays'
    ELSE t.SalesTerritoryCountry END AS pays,
    SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (d.CalendarYear, t.SalesTerritoryCountry)
ORDER BY GROUPING(d.CalendarYear), d.CalendarYear,
         GROUPING(t.SalesTerritoryCountry), t.SalesTerritoryCountry;
```

Observez. Plus aucun NULL ambigu : les lignes de synthèse disent « Toutes années » et « Tous pays ». Le tri par GROUPING(...) d'abord range proprement les sous-totaux sous leur détail.

2.3 — Agrégats conditionnels : un continent par colonne

Théorie : Theorie_06.md, section 6.

Objectif : les ventes par année, avec une colonne par continent.

```
SELECT d.CalendarYear AS annee,
    SUM(CASE WHEN t.SalesTerritoryGroup = 'North America' THEN f.SalesAmount END) AS americque_nord,
    SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe' THEN f.SalesAmount END) AS europe,
    SUM(CASE WHEN t.SalesTerritoryGroup = 'Pacific' THEN f.SalesAmount END) AS pacifique
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear
ORDER BY annee;
```

Observez. Une année par ligne, trois continents en colonnes. Le CASE placé dans le SUM ne retient que les ventes du bon continent ; les autres deviennent NULL et sont ignorées.

2.4 — Le même tableau croisé avec PIVOT, et le repli par UNPIVOT

Théorie : Theorie_06.md, section 7.

Objectif : obtenir le résultat de 2.3 avec la syntaxe dédiée, puis refaire le chemin inverse.

```
SELECT *
FROM (
    SELECT d.CalendarYear AS annee, t.SalesTerritoryGroup AS continent,
           f.SalesAmount AS montant
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
) AS src
PIVOT (SUM(montant) FOR continent IN ('North America', 'Europe', 'Pacific')) AS p
ORDER BY annee;
```

Observez. Même tableau qu'en 2.3, mais sans répéter un CASE par colonne. Rappel des deux règles : la sous-requête src ne contient que annee, continent, montant ; et les valeurs pivotées sont écrites en dur.

Ajoutez maintenant f.ProductKey dans la sous-requête src, sans rien changer d'autre, et relancez. Le résultat éclate en une ligne par produit et par année : la colonne en trop est entrée dans le regroupement implicite. C'est le piège numéro un du PIVOT, et il ne produit aucune erreur, seulement un résultat au mauvais grain.

Passons au chemin inverse. Pour avoir un tableau large à replier, et une case vide dedans, croisez cette fois les quatre premiers pays plutôt que les continents, et gardez le résultat sous la main :

```
CREATE OR REPLACE TEMP TABLE croise_pays AS
SELECT *
FROM (
    SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
           f.SalesAmount AS montant
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
) AS src
PIVOT (SUM(montant) FOR pays IN ('Australia', 'Canada', 'France', 'Germany')) AS p
ORDER BY annee;

SELECT * FROM croise_pays;
```

Observez. Cinq années sur quatre pays, et une case vide : l'Allemagne n'a aucune vente en 2010. Vingt cases, dont dix-neuf remplies. Repliez maintenant ce tableau :

```
UNPIVOT croise_pays
ON 'Australia', 'Canada', 'France', 'Germany'
```

```
INTO NAME pays VALUE ventes
ORDER BY annee, pays;
```

Observez, et comptez. Dix-neuf lignes, pas vingt : le couple 2010 et Germany a disparu. UNPIVOT écarte les valeurs nulles, sans le dire et sans erreur. C'est son piège silencieux, et il mord exactement là où on se sert de lui, sur un tableau large reçu d'ailleurs dont on ne connaît pas les trous. Si vous repliez ensuite pour compter les couples, ou pour joindre, il vous manquera des lignes.

DuckDB n'offre pas d'option pour les garder : c'est un COALESCE par colonne qui fait le travail.

```
UNPIVOT croise_pays
ON COALESCE("Australia", 0) AS 'Australia', COALESCE("Canada", 0) AS 'Canada',
   COALESCE("France", 0) AS 'France', COALESCE("Germany", 0) AS 'Germany'
INTO NAME pays VALUE ventes
ORDER BY annee, pays;
```

Observez. Vingt lignes, et l'Allemagne de 2010 à zéro. Mesurez bien ce que vous venez de décider : la ligne existe maintenant, mais elle affirme une vente nulle là où la donnée disait « pas de donnée ». Pour compter des couples, c'est ce qu'on veut ; pour calculer une moyenne par pays, c'est un faux zéro qui la tire vers le bas, et il valait mieux garder le trou (théorie, section 6).

Portabilité — Snowflake écarte lui aussi les valeurs nulles par défaut, d'après sa documentation, et offre en plus les mots `EXCLUDE NULLS` et `INCLUDE NULLS` pour le dire dans la requête. DuckDB ne connaît ni l'un ni l'autre : les écrire rend une erreur de syntaxe, mesurée. Le comportement par défaut est donc le même sur les deux, mais seul l'un des deux se laisse contredire.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_06.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vous produisez maintenant des rapports complets, avec le détail, les sous-totaux et les tableaux croisés. Le Cours 07 change d'angle : les sous-requêtes corrélées et les opérateurs d'ensembles (`EXISTS`, `IN`, `UNION`, `INTERSECT`, `EXCEPT`), pour poser des questions que l'agrégation seule ne sait pas exprimer.